



GRAMÁTICA BNF

GoLite Compilador – Fase 1

Organización de Lenguajes y Compiladores 1

Universidad San Carlos de Guatemala – FIUSAC

Escuela de Vacaciones Junio

Yerri Jonatan Gadiel Bamaca Lopez

202132066

1. PROGRAMA

$\langle \text{programa} \rangle ::= \langle \text{declaraciones_globales} \rangle$

$\langle \text{declaraciones_globales} \rangle$

$::= \langle \text{declaraciones_globales} \rangle \langle \text{declaracion_global} \rangle$

$| \epsilon$

$\langle \text{declaracion_global} \rangle ::= \langle \text{decl_funcion} \rangle$

2. FUNCIONES

$\langle \text{decl_funcion} \rangle ::= \text{func ID (} \langle \text{lista_parametros} \rangle \text{) } \langle \text{tipo} \rangle \langle \text{bloque} \rangle$

$| \text{func ID (} \langle \text{lista_parametros} \rangle \text{) } \langle \text{bloque} \rangle$

$\langle \text{lista_parametros} \rangle ::= \langle \text{lista_parametros_ne} \rangle$

$| \epsilon$

$\langle \text{lista_parametros_ne} \rangle ::= \langle \text{lista_parametros_ne} \rangle , \langle \text{parametro} \rangle$

$| \langle \text{parametro} \rangle$

$\langle \text{parametro} \rangle ::= \text{ID } \langle \text{tipo} \rangle$

3. TIPOS

<tipo> ::= int
| float64
| string
| bool
| rune
| ID

4. BLOQUES Y SENTENCIAS

<bloque> ::= { <sentencias> }

<sentencias> ::= <sentencias> <sentencia>
| ϵ

<sentencia> ::= <sentencia_simple> ;
| <sentencia_simple>
| <sentencia_if>
| <sentencia_for>
| <bloque>

<sentencia_simple> ::= <decl_var>
| <asignacion>
| <asignacion_compuesta>
| <inc_dec>
| <sentencia_break>
| <sentencia_continue>
| <sentencia_llamada>

5. DECLARACIÓN DE VARIABLES

<decl_var> ::= var ID <tipo> = <expresion>
| var ID <tipo>
| ID := <expresion>

6. ASIGNACIÓN

<asignacion> ::= ID = <expresion>

<asignacion_compuesta> ::= ID += <expresion>
| ID -= <expresion>

<inc_dec> ::= ID ++
| ID --

7. CONTROL DE FLUJO

<sentencia_if> ::= if <expresion> <bloque>
 | if <expresion> <bloque> else <bloque>
 | if <expresion> <bloque> else <sentencia_if>
 | if (<expresion>) <bloque>
 | if (<expresion>) <bloque> else <bloque>
 | if (<expresion>) <bloque> else <sentencia_if>

<sentencia_for> ::= for <sentencia_simple> ; <expresion> ; <sentencia_simple>
<bloque>
 | for <expresion> <bloque>

<sentencia_break> ::= break

<sentencia_continue> ::= continue

8. LLAMADAS A FUNCIONES

<sentencia_llamada> ::= <llamada_funcion>
 | <llamada_embebida>

<llamada_funcion> ::= ID (<lista_args>)

<lista_args> ::= <lista_args_ne>

| ϵ

<lista_args_ne> ::= <lista_args_ne> , <expresion>
| <expresion>

<llamada_embebida> ::= fmt.Println (<lista_args>)
| strconv.Atoi (<expresion>)
| strconv.ParseFloat (<expresion>)
| reflect.TypeOf (<expresion>)

sentencia_continue

::= CONTINUE:c
{: RESULT = new NodoContinue(cleft, cright); :}
;

/* Sentencia llamada consolidada */

sentencia_llamada

::= llamada_funcion:l {: RESULT = new NodoExpresionSentencia(l, lleft, lright); :}
| llamada_funcion_compuesta:l {: RESULT = new NodoExpresionSentencia(l, lleft, lright); :}
| llamada_embebida:e {: RESULT = new NodoExpresionSentencia(e, eleft, eright); :}
;

llamada_funcion

::= ID:nombre PAREN_ABRE lista_args:args PAREN_CIERRA

```

    { : RESULT = new NodoLlamadaFuncion(nombre.getLexema(), args, nombreleft,
nombreright); :}

;

```

llamada_funcion_compuesta

```

::= ID:paquete PUNTO ID:metodo PAREN_ABRE lista_args:args PAREN_CIERRA

    { : RESULT = new NodoLlamadaFuncion(paquete.getLexema() + "." +
metodo.getLexema(), args, paqueteleft, paqueteright); :}

;

```

9. EXPRESIONES (precedencia de mayor a menor)

```

<expresion>      ::= <expresion_or>

```

```

<expresion_or>   ::= <expresion_or> || <expresion_and>
                  | <expresion_and>

```

```

<expresion_and>  ::= <expresion_and> && <expresion_igualdad>
                  | <expresion_igualdad>

```

```

<expresion_igualdad> ::= <expresion_igualdad> == <expresion_relacional>
                  | <expresion_igualdad> != <expresion_relacional>
                  | <expresion_relacional>

```

expresion_relacional

```

::= expresion_relacional:izq MENOR expresion_suma:der

```

```

    {: RESULT = new NodoBinario("<", izq, der, izqleft, izqright); :}
| expresion_relacional:izq MAYOR expresion_suma:der
    {: RESULT = new NodoBinario(">", izq, der, izqleft, izqright); :}
| expresion_relacional:izq MENOR_IGUAL expresion_suma:der
    {: RESULT = new NodoBinario("<=", izq, der, izqleft, izqright); :}
| expresion_relacional:izq MAYOR_IGUAL expresion_suma:der
    {: RESULT = new NodoBinario(">=", izq, der, izqleft, izqright); :}
| expresion_suma:e {: RESULT = e; :}
;

```

expresion_suma

```

::= expresion_suma:izq SUMA expresion_mult:der
    {: RESULT = new NodoBinario("+", izq, der, izqleft, izqright); :}
| expresion_suma:izq RESTA expresion_mult:der
    {: RESULT = new NodoBinario("-", izq, der, izqleft, izqright); :}
| expresion_mult:e {: RESULT = e; :}
;

```

expresion_mult

```

::= expresion_mult:izq MULT expresion_unaria:der
    {: RESULT = new NodoBinario("*", izq, der, izqleft, izqright); :}
| expresion_mult:izq DIV expresion_unaria:der
    {: RESULT = new NodoBinario("/", izq, der, izqleft, izqright); :}
| expresion_mult:izq MOD expresion_unaria:der
    {: RESULT = new NodoBinario("%", izq, der, izqleft, izqright); :}
| expresion_unaria:e {: RESULT = e; :}
;

```


expresion_unaria

```
::= RESTA:r expresion_unaria:e
    {: RESULT = new NodoUnario("-", e, rleft, rright); :}
    %prec UMINUS
| NOT:n expresion_unaria:e
    {: RESULT = new NodoUnario("!", e, nleft, nright); :}
| primario:p {: RESULT = p; :}
;
```

primario

```
::= ENTERO:t
    {: RESULT = new NodoLiteral(Integer.parseInt(t.getLexema()), "int", tleft, tright); :}
| FLOTANTE:t
    {: RESULT = new NodoLiteral(Double.parseDouble(t.getLexema()), "float64", tleft, tright); :}
| STRING:t
    {:
        String raw = t.getLexema();
        String val = raw.substring(1, raw.length()-1);
        RESULT = new NodoLiteral(val, "string", tleft, tright);
    :}
| RUNE_LIT:t
    {:
        String raw = t.getLexema();
        char val = raw.charAt(1);
```

```

    if (raw.charAt(1) == '\\') {
        switch (raw.charAt(2)) {
            case 'n': val = '\n'; break;
            case 't': val = '\t'; break;
            case 'r': val = '\r'; break;
            case '\\': val = '\\'; break;
            case '"': val = '"'; break;
            case "'": val = "'"; break;
        }
    }

    RESULT = new NodoLiteral((int)val, "rune", tleft, tright);
:}

| BOOLEANO:t
{: boolean val = t.getLexema().equals("true");
    RESULT = new NodoLiteral(val, "bool", tleft, tright);
:}

| NIL:t
{: RESULT = new NodoLiteral(null, "nil", tleft, tright); :}

| ID:t
{: RESULT = new NodoIdentificador(t.getLexema(), tleft, tright); :}

| PAREN_ABRE expresion:e PAREN_CIERRA
{: RESULT = e; :}

| llamada_funcion:l {: RESULT = l; :}

| llamada_funcion_compuesta:l {: RESULT = l; :}

| llamada_embebida:l {: RESULT = l; :}

;

```

10. TOKENS TERMINALES

Literales:

ENTERO ::= [0-9]+

FLOTANTE ::= [0-9]+ "." [0-9]+

STRING ::= "'" ([^"\\] | \\.)* "'"

RUNE_LIT ::= "\"" ([^'\\] | \\[nrt\\"])* "\""

BOOLEANO ::= "true" | "false"

NIL ::= "nil"

Tipos:

int float64 string bool rune

Palabras reservadas:

var func if else for switch case default

break continue return range struct

Funciones embebidas (tokens compuestos):

fmt.Println strconv.Atoi strconv.ParseFloat reflect.TypeOf

terminal Token ENTERO, FLOTANTE, STRING, RUNE_LIT, BOOLEANO, NIL;

terminal Token TIPO_INT, TIPO_FLOAT64, TIPO_STRING, TIPO_BOOL, TIPO_RUNE;

terminal Token VAR, FUNC, IF, ELSE, FOR;

terminal Token BREAK, CONTINUE;

terminal Token RETURN;

terminal Token PUNTO;

terminal Token FMT_PRINTLN, STRCONV_ATOI, STRCONV_PARSEFLOAT,
REFLECT_TYPEOF;

terminal Token ID;

terminal Token SUMA, RESTA, MULT, DIV, MOD;

terminal Token ASIGNACION, ASIGN_CORTO, ASIGN_SUMA, ASIGN_RESTA;

terminal Token IGUAL, DIFERENTE, MENOR, MAYOR, MENOR_IGUAL, MAYOR_IGUAL;

terminal Token AND, OR, NOT;

terminal Token INCREMENTO, DECREMENTO;

terminal Token UMINUS;

terminal Token PAREN_ABRE, PAREN_CIERRA;

terminal Token LLAVE_ABRE, LLAVE_CIERRA;

terminal Token PUNTO_COMA, COMA;

Operadores:

+ - * / % = := += -= == != < > <= >= && || ! ++ --

Delimitadores:

() { } [] ; : , .

Comentarios (ignorados por el lexer):

// comentario de línea

/* comentario de bloque */

TABLA DE PRECEDENCIA Y ASOCIATIVIDAD (mayor → menor prioridad)

Nivel	Operadores	Asociatividad
1	() []	izq → der
2	! - (unario)	der → izq
3	* / %	izq → der
4	+ -	izq → der
5	< <= > >=	izq → der
6	== !=	izq → der
7	&&	izq → der
8		izq → der

Tabla de Precedencia

La siguiente tabla resume la prioridad de los operadores implementados en el analizador sintáctico

Nivel	Operadores	Asociatividad
1	() . (acceso)	Izquierda → Derecha

Nivel	Operadores	Asociatividad
2	! - (unario)	Derecha → Izquierda
3	* / %	Izquierda → Derecha
4	+ -	Izquierda → Derecha
5	< <= > >=	Izquierda → Derecha
6	== !=	Izquierda → Derecha
7	&&	Izquierda → Derecha
8	`	